

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

Big Data Analytics

Submitted by

Sinchana Hemanth (1BM23CS330)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Big Data Analytics” carried out by **Sinchana Hemanth (1BM23CS330)**, who is a bona fide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2026. The Lab report has been approved as it satisfies the academic requirements in respect of a **Big Data Analytics - (23CS6PCBDA)** work prescribed for the said degree.

Prof. Supreeth S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

INDEX SHEET

Sl. No.	Experiment Title	Page No.
1.	MongoDB CRUD Operations	1-8
2.	Cassandra Employee	9-13
3.	Cassandra Library	14-21
4.	Hadoop Commands	22-24
5.	Hadoop Word Count	25-29
6.	Hadoop Weather program	30-40
7.	Hadoop Top N Program	41-46
8.	Scala Program	47-50
9.	Spark program	47-50
10.	Open Ended question	51-56

Course Outcome

CO1	Apply the concepts of NoSQL, Hadoop, Spark for a given task
CO2	Analyze data analytic techniques for a given problem
CO3	Conduct experiments using data analytics mechanisms for a given problem

Github link: <https://github.com/SinchanaHemanth/SinchanaHemanth-1BM23CS330-BDAlab.git>

LAB 1

MongoDB - CRUD Demonstration

Student database

1. Create a database “Student” with the following attributes Rollno, Age, ContactNo, Email-Id.

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.createCollection("Student");
{ ok: 1 }
```

2. Insert appropriate values (at least 5

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.insert({RollNo:1, Age:21, Cont:9876, email:"antara.de9@gmail.com"});
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.insert({RollNo:10, Age:23, Cont:2276, email:"rekha.de9@gmail.com"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a82f47c840f42b4a05534") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.insert({RollNo:2, Age:22, Cont:9976, email:"anushka.de9@gmail.com"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a82ed7c840f42b4a05531") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.insert({RollNo:3, Age:21, Cont:5576, email:"anubhav.de9@gmail.com"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a82ed7c840f42b4a05532") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.insert({RollNo:4, Age:20, Cont:4476, email:"pani.de9@gmail.com"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a82ed7c840f42b4a05533") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.insert({RollNo:10, Age:23, Cont:2276, email:"rekha.de9@gmail.com"});
{
  acknowledged: true,
```

3. View the data

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.find()
[
  {
    _id: ObjectId("660a82ec7c840f42b4a05530"),
    RollNo: 1,
    Age: 21,
    Cont: 9876,
    email: 'antara.de9@gmail.com'
  },
  {
    _id: ObjectId("660a82ed7c840f42b4a05531"),
    RollNo: 2,
    Age: 22,
    Cont: 9976,
    email: 'anushka.de9@gmail.com'
  },
  {
    _id: ObjectId("660a82ed7c840f42b4a05532"),
    RollNo: 3,
    Age: 21,
    Cont: 5576,
    email: 'anubhav.de9@gmail.com'
  },
  {
    _id: ObjectId("660a82ed7c840f42b4a05533"),
    RollNo: 4,
    Age: 20,
    Cont: 4476,
    email: 'pani.de9@gmail.com'
  },
  {
    _id: ObjectId("660a82f47c840f42b4a05534"),
    RollNo: 10,
    Age: 23,
    Cont: 2276,
    email: 'rekha.de9@gmail.com'
  }
]
```

4. Write a query to update the Email-Id of a student with rollno 10.

```
{
  _id: ObjectId("660a83337c840f42b4a05535"),
  RollNo: 11,
  Age: 22,
  Name: 'ABC',
  Cont: 2276,
  email: 'rea.de9@gmail.com'
}
```

5. Replace the student's name from "ABC" to "FEM" of rollno 11.

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.update({RollNo:11,Name:"ABC"},{$set:{Name:"FEM"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

6. Export the created table into local file system

```
C:\Users\BMSCECSE>mongoexport mongodb+srv://antararc:Test1234@cluster0.mfnfeys.mongodb.net/myDB --collection=Student --out C:\Users\BMSCECSE\Downloads\output.json
2023-01-12T15:15:56.383+0530 connected to: mongodb+srv://[**REDACTED**]@cluster0.mfnfeys.mongodb.net/myDB
2023-01-12T15:15:56.497+0530 exported 7 records
```

7. Drop the table

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Student.drop();
true
```

8. Import a given csv dataset from local file system into mongodb collection.

```
C:\Users\BMSCECSE>mongoimport mongodb+srv://antararc:Test1234@cluster0.mfnfeys.mongodb.net/myDB --collection=New_Student --type json --file C:\Users\BMSCECSE\Downloads\output.json
2023-01-12T15:17:35.523+0530 connected to: mongodb+srv://[**REDACTED**]@cluster0.mfnfeys.mongodb.net/myDB
2023-01-12T15:17:35.640+0530 7 document(s) imported successfully. 0 document(s) failed to import.
```

Customer database

1. Create a collection by name Customers with the following attributes.

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.createCollection("Customers");
{ ok: 1 }
```

Cust_id, Acc_Bal, Acc_T

2. Insert at least 5 values into the table

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.insert({cust_id:1,Balance:200, Type:"S"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a83b47c840f42b4a05536") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.insert({cust_id:1,Balance:1000, Type:"Z"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a83b47c840f42b4a05537") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.insert({cust_id:2,Balance:100, Type:"Z"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a83b47c840f42b4a05538") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.insert({cust_id:2,Balance:1000, Type:"C"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a83b57c840f42b4a05539") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.insert({cust_id:2,Balance:500, Type:"C"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a83b57c840f42b4a0553a") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.insert({cust_id:2,Balance:50, Type:"S"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a83b57c840f42b4a0553b") }
}
Atlas atlas-b6pfyk-shard-0 [primary] test>
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.insert({cust_id:3,Balance:500, Type:"Z"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a83b77c840f42b4a0553c") }
}
```

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.insert({cust_id:2,Balance:50, Type:"S"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("660a83b57c840f42b4a0553b") }
}
```

3. Write a query to display those records whose total account balance is greater than 1200 of account type 'Z' for each customer_id.

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.aggregate (
...  {$match:{Type:"Z"}},
...
...  {$group : { _id : "$cust_id",
...
...  TotAccBal : {$sum:"$Balance"} } },
...  {$match:{TotAccBal:{$gt:1200}}});
```

4. Determine Minimum and Maximum account balance for each customer_id.

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.aggregate (
...
... {$group : { _id : "$cust_id",
...
... minAccBal :{$min:"$Balance"},
... maxAccBal :{$max:"$Balance"} }});
[
  { _id: 2, minAccBal: 50, maxAccBal: 1000 },
  { _id: 1, minAccBal: 200, maxAccBal: 1000 },
  { _id: 3, minAccBal: 500, maxAccBal: 500 }
]
```

5. Drop the table

```
Atlas atlas-b6pfyk-shard-0 [primary] test> db.Customers.drop()
true
```


LAB 1 : SQL

create database company;

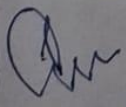
use company;

create table employees (employee_id INT PRIMARY KEY, name varchar(50),
department_id INT, salary INT, hire_date DATE, manager_id INT,
FOREIGN KEY (department_id) REFERENCES departments(department_id));

insert into employees values (1, 'Alice', 2, 70000, '2021-03-15', NULL);

select * from employees;

1. select name from employees where hire_date > '2020-01-01';
2. select name from employees where name like 'A%';
3. select name from employees where department_id IS NULL;
4. select * from employees ~~order~~ order by salary desc limit 5;
5. select * from employees where year(hire_date) = '2022';
6. select department_id, count(employee_id) from employees group by department_id;
7. select department_id from employees group by department_id having count(*) > 3;
8. select e.name, d.department_id, d.name from departments as d join employees as e on e.department_id = d.department_id;
9. select e.name, m.name from employees as e join employees as m on ~~m~~ e.manager_id = m.employee_id;


2/2/20

LAB-2 MongoDB

1. Create database called Employee.

use Employee

2. Create collection Employee-Info

db.createCollection("Employee-Info")

3. Insert values into collection

db.Employee-Info.insertMany([

{ Emp-Id: 120,

Emp-Name: "Rahul",

Designation: "Developer",

Date-of-Joining: ISODate("2020-06-15"),

Salary: 60000,

Dept-Name: "IT",

},

{ Emp-Id: 121,

Emp-Name: "Anitha",

Designation: "Analyst",

Date-of-Joining: ISODate("2019-04-10"),

Salary: 55000,

Dept-Name: "Finance"

}]])

4. Update Employee name and Department of Emp-Id 121

db.Employee-Info.updateOne (

{ Emp-Id: 121 },

{ \$set: { Emp-Name: "Anitha Sharma", Dept-Name: "HR" }

})

5. Sort Employee based on salary

db.Employee-Info.find().sort({ Salary: -1 })

7. Update the table to add project names

```
db.Employee-Info.updateOne(  
  { Emp-Id: 120 },  
  { $set: { Projects: ["Payroll System", "Employee Portal"] } })
```

8. Create TTL of 15 seconds to display employee value

Step 1:

```
db.Employee-Info.updateMany( {}, { $set: { expiresAt: new Date() } })
```

Step 2:

```
db.Employee-Info.createIndex( { expiresAt: 1 },  
  { expireAfterSeconds: 15 })
```

Cassandra

1. Create keyspace by name Employee

```
CREATE KEYSPACE Employee WITH REPLICATION =  
  { 'class': 'simpleStrategy', 'replication_factor': 1 };
```

2. List all keyspaces

```
DESCRIBE KEYSPACES
```

3. Switch to Keyspace

```
USE Employee;
```

4. Create Column Family Employee-Info

```
CREATE TABLE Employee-Info (Emp-Id int PRIMARY KEY,  
  Emp-Name text, Designation text, Date-of-Joining timestamp,  
  Salary double, Dept-Name text);
```

5. List all tables to check if Employee-Info table is created

```
DESCRIBE TABLES;
```

6. Insert values into table using BATCH

```
BEGIN BATCH  
INSERT INTO Employee-Info (Emp-Id, Emp-Name, Designation,  
  Date-of-Joining, Salary, Dept-Name) VALUES (121, 'Amit',  
  'Software Engineer', '2019-06-15', 60000, 'IT');
```


LAB 2

Perform the following DB operations using Cassandra

Employee Database

1. Create a keyspace by name Employee

```
cqlsh:library> CREATE KEYSPACE Employee WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor' : 1 };
cqlsh:library>
```

2. Create a column family by name Employee-Info with attributes Emp_Id Primary Key,

```
cqlsh:employee>
cqlsh:employee> CREATE TABLE Employee_Info (
...     Emp_Id int PRIMARY KEY,
...     Emp_Name text,
...     Designation text,
...     Date_of_Joining date,
...     Salary decimal,
...     Dept_Name text
... );
```

Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name

3. Insert the values into the table in batch

```
cqlsh:employee> BEGIN BATCH
... INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
... VALUES (101, 'John Doe', 'Manager', '2023-01-01', 50000, 'HR');
... INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
... VALUES (121, 'Jane Smith', 'Developer', '2023-02-01', 60000, 'IT');
APPLY BATCH;
cqlsh:employee> UPDATE Employee_Info SET Emp_Name = 'Jane Johnson', Dept_Name = 'Engineering' WHERE Emp_Id = 121;
cqlsh:employee> SELECT * FROM Employee_Info;
```

emp_id	date_of_joining	dept_name	designation	emp_name	salary
121	2023-02-01	Engineering	Developer	Jane Johnson	60000
101	2023-01-01	HR	Manager	John Doe	50000

(2 rows)

4. Update Employee name and Department of Emp-Id 121

5. Sort the details of Employee records based on salary

```
cqlsh:employee> paging off
Disabled Query paging.
cqlsh:employee> SELECT * FROM Employee_Info WHERE Emp_Id IN (121,101) ORDER BY Salary ALLOW FILTERING;

emp_id | salary | date_of_joining | dept_name | designation | emp_name
-----+-----+-----+-----+-----+-----
121    | 60000 | 2023-02-01      | IT        | Developer   | Jane Smith
101    | 50000 | 2023-01-01      | HR        | Manager     | John Doe

cqlsh:employee> UPDATE Employee_Info SET Projects = {'ProjectA', 'ProjectB'} WHERE Emp_Id = 101 and salary=50000;
cqlsh:employee> UPDATE Employee_Info SET Projects = {'ProjectC'} WHERE Emp_Id = 121 and salary=60000;
cqlsh:employee> select * from Employee_Info;
```

emp_id	salary	date_of_joining	dept_name	designation	emp_name	projects
121	60000	2023-02-01	IT	Developer	Jane Smith	{'ProjectC'}
101	50000	2023-01-01	HR	Manager	John Doe	{'ProjectA', 'ProjectB'}

(2 rows)

6. Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee.

7. Update the altered table to add project names.

```
cqlsh:employee> UPDATE Employee_Info SET Projects = {'ProjectA', 'ProjectB'} WHERE Emp_Id = 101 and salary=50000;
cqlsh:employee> UPDATE Employee_Info SET Projects = {'ProjectC'} WHERE Emp_Id = 121 and salary=60000;
cqlsh:employee> select * from Employee_Info;
```

emp_id	salary	date_of_joining	dept_name	designation	emp_name	projects
121	60000	2023-02-01	IT	Developer	Jane Smith	{'ProjectC'}
101	50000	2023-01-01	HR	Manager	John Doe	{'ProjectA', 'ProjectB'}

(2 rows)

8. Create a TTL of 15 seconds to display the values of Employees.

```
cqlsh:employee> INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name) VALUES (102, 'Jane Smith', 'Developer', '2022-06-03', 60000, 'IT') USING TTL 15;  
cqlsh:employee> select ttl(Emp_Name) from Employee_Info where Emp_id=102;  
  
ttl(emp_name)  
-----  
14  
  
(1 rows)
```

Cassandra

1. Create keyspace by name Employee

```
CREATE KEYSPACE Employee WITH REPLICATION =  
{ 'class': 'SimpleStrategy', 'replication_factor': 1 };
```

2. List all keyspaces

```
DESCRIBE KEYSPACES
```

3. Switch to Keyspace

```
USE Employee;
```

4. Create Column Family Employee-Info

```
CREATE TABLE Employee-Info (Emp-Id int PRIMARY KEY,  
Emp-Name text, Designation text, Date-of-Joining timestamp,  
Salary double, Dept-Name text);
```

5. List all tables to check if Employee-Info table is created

```
DESCRIBE TABLES;
```

6. Insert values into table using BATCH

```
BEGIN BATCH  
INSERT INTO Employee-Info (Emp-Id, Emp-Name, Designation,  
Date-of-Joining, Salary, Dept-Name) VALUES (101, 'Amil',  
'Software Engineer', '2019-06-15', 60000, 'IT');
```

7. Update Employee Name and Department of Emp-Id 121

```
UPDATE Employee-Info SET Emp-Name = Amit, Dept-Name =  
'Data Science' WHERE Emp-Id = 121;
```

8. Sort details of Employee records based on salary

```
CREATE TABLE Employee-By-Salary (Dept-Name text, salary int,  
Emp-Id int, Emp-Name text, PRIMARY KEY (Dept-Name, salary,  
Emp-Id)) WITH CLUSTERING ORDER BY (salary DESC);
```

```
SELECT * FROM Employee-By-Salary WHERE Dept-Name = "IT";
```

~~2/2/26~~

LAB 3

Perform the following DB operations using Cassandra

1. Create a keyspace by name Library

```
cqlsh> CREATE KEYSPACE Library WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor' : 1 };
cqlsh> show keyspaces;
Improper show command.
cqlsh> use Library;
cqlsh:library> |
```

2. Create a column family by name Library-Info with attributes

Stud_Id Primary Key, Counter_value of type Counter,

Stud_Name, Book-Name, Book-Id, Date_of_issue

```
cqlsh:library> CREATE TABLE Library_Info (Stud_Id int PRIMARY KEY,Counter_value counter,Stud_Name text,Book_Name text,Book_Id text,Date_of_issue timestamp);
InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot mix counter and non counter columns in the same table"
cqlsh:library> CREATE TABLE Library_Info (
...     Stud_Id int PRIMARY KEY,
...     Stud_Name text,
...     Book_Name text,
...     Book_Id text,
...     Date_of_issue timestamp
... );
cqlsh:library> CREATE TABLE Library_Counters (
...     Stud_Id int PRIMARY KEY,
...     Counter_value counter
... );
cqlsh:library>
```

3. Insert the values into the table in batch

```
cqlsh:library> BEGIN BATCH
... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue) VALUES (112, 'John Doe', 'BDA', 'B001', '2023-01-01');
... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue) VALUES (113, 'Jane Smith', 'ML', 'B002', '2023-01-02');
... APPLY BATCH;
```

4. Display the details of the table created and increase the value of the counter

```
cqlsh:library> SELECT * FROM Library_Info;

stud_id | book_id | book_name | date_of_issue | stud_name
-----+-----+-----+-----+-----
113 | B002 | ML | 2023-01-02 00:00:00.000000+0000 | Jane Smith
112 | B001 | BDA | 2023-01-01 00:00:00.000000+0000 | John Doe
(2 rows)
cqlsh:library> SELECT * FROM Library_Counters;

stud_id | counter_value
-----+-----
113 | 1
112 | 1
(2 rows)
```

5. Write a query to show that a student with id 112 has taken a book “BDA” 2 times.

```
cqlsh:library> UPDATE Library_Counters SET Counter_value = Counter_value + 1 WHERE Stud_Id = 112;
cqlsh:library> SELECT * FROM Library_Counters WHERE Stud_Id = 112;

stud_id | counter_value
-----+-----
    112 |             2
(1 rows)
```

6. Export the created column to a csv file

```
cqlsh:library> COPY Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue) TO 'file.csv' WITH HEADER = TRUE;
Using 11 child processes

Starting copy of library.library_info with columns [stud_id, stud_name, book_name, book_id, date_of_issue].
Processed: 2 rows; Rate:      10 rows/s; Avg. rate:      6 rows/s
2 rows exported to 1 files in 0.374 seconds.
cqlsh:library> COPY Library_Counters (Stud_Id, Counter_value) FROM 'library_counters.csv' WITH HEADER = TRUE;
Using 11 child processes
```

7. Import a given csv dataset from local file system into Cassandra column family

```
cqlsh:library> copy library_info(Stud_Id,Stud_Name,Book_Name,Book_Id,Date_of_issue) from 'file.csv' with header=true;
Using 7 child processes

Starting copy of library.library_info with columns [stud_id, stud_name, book_name, book_id, date_of_issue].
Processed: 2 rows; Rate:      2 rows/s; Avg. rate:      4 rows/s
2 rows imported from 1 files in 0.513 seconds (0 skipped).
cqlsh:library> select * from library_info;

stud_id | book_id | book_name | date_of_issue | stud_name
-----+-----+-----+-----+-----
    113 |    B002 |      ML | 2023-01-02 00:00:00.000000+0000 | Jane Smith
    112 |    B001 |      BDA | 2023-01-01 00:00:00.000000+0000 | John Doe
```


LAB-3

Questions:

- ① (a) Create keyspace by name library
- (b) Create column ~~name~~ family by name library-Info with attributes
- (c) Create column family by name library-Info with attributes Stud-Id primary key, Counter-value of type Counter, Stud-Name, Book-name, Book-Id, Date-of-Issue
- (d) Insert values into table in batch
- (e) Display details of table created and increase value of counter
- (f) Write query to show that student with ID 112 has taken book "BDA" 2 times
- (g) Export created column to a csv file
- (h) Import given csv dataset from local file system into Cassandra column family

Answers

```
(a) CREATE KEYSPACE library
WITH replication = {
  'class' : 'SimpleStrategy',
  'replication-factor' : 1
};
```

```
(b) USE library;
```

```
(c) CREATE TABLE library-Info (
  Stud-Id int,
  Stud-Name text,
  Book-Name text,
  Book-Id int,
  Date-of-Issue date,
  PRIMARY KEY (Stud-Id, Book-Id)
);
```

```
CREATE TABLE Book-Counter (
    Stud-Id int,
    Book-Name text,
    Counter-Value counter,
    PRIMARY KEY (Stud-Id, Book-Name)
);
```

(c) BEGIN BATCH

```
INSERT INTO Library-Info (Stud-Id, Stud-Name, Book-Name,
    Book-Id, Date-of-issue) VALUES (112, 'Rahul', 'BDA',
    101, '2026-02-10');
```

```
INSERT INTO Library-Info (Stud-Id, Stud-Name, Book-Name,
    Book-Id, Date-of-issue) VALUES (113, 'Anie', 'DBMS',
    102, '2026-02-12');
```

(d) UPDATE Book-Counter

```
SET Counter-Value = Counter-Value + 1
```

```
WHERE Stud-Id = 112 AND Book-Name = 'BDA';
```

(e) DISPLAY TABLE Library-Info

```
DISPLAY * FROM Library-Info
```

(f) SELECT Counter-Value

```
FROM Book-Counter
```

```
WHERE Stud-Id = 112 AND Book-Name = 'BDA';
```

(g) COPY Library-Info TO 'C:/Library-info.csv' WITH HEADER
= TRUE;

(h) COPY Library-Info (Stud-Id, Stud-Name, Book-Name,
Book-Id, Date-of-issue) FROM

```
'C:/Library-info.csv' WITH HEADER = TRUE;
```

LAB-4

① Student database

(a) ~~create keyspace~~ create keyspace

```
CREATE KEYSPACE college  
WITH replication = { 'class': 'SimpleStrategy', 'replication_factor':  
1 };
```

(b) create table student_info with : USN, Name, Branch, Semester, CGPA

USE college;

```
CREATE TABLE student_info (USN, text PRIMARY KEY, Name  
text, Branch text, Semester int, CGPA float);
```

(c) Insert atleast 3 student records

```
INSERT INTO student_info (USN, Name, Branch, Semester, CGPA)  
VALUES ('1CS01', 'Arun', 'CSE', 5, 8.2);
```

```
INSERT INTO student_info (USN, Name, Branch, Semester, CGPA)  
VALUES ('1CS02', 'Priya', 'CSE', 5, 8.8);
```

```
INSERT INTO student_info (USN, Name, Branch, Semester, CGPA)  
VALUES ('1CS03', 'Rahul', 'ECE', 4, 7.9);
```

(d) ~~UPDATE~~ Update the CGPA of student with given ~~USN~~ USN

```
UPDATE student_info SET CGPA = 9.0 WHERE USN = '1BM23CS  
330';
```

(e) Display all records

```
SELECT * FROM student_info
```


② Book management system

(a) Create keyspace named library

```
CREATE KEYSPACE library WITH replication =  
[ 'class': 'SimpleStrategy', 'replication-factor': 1 ];
```

(b) Create table Book-Info with attributes

```
USE library;  
CREATE TABLE Book-Info (Book-Id int PRIMARY KEY,  
Title text, Author text, Price float, Category text);
```

(c) Insert records using batch

```
BEGIN BATCH
```

```
INSERT INTO  
INSERT INTO Book-Info (Book-Id, Title, Author, Price,  
Category) VALUES (1, 'DBMS', 'Russell', 500,  
'Education');
```

(d) Update price of specific book

```
UPDATE Book-Info SET Price = 550 WHERE Book-Id  
= 1;
```

(e) Delete Book record using Book-Id

```
DELETE FROM Book-Info WHERE Book-Id = 2;
```

③ Customer orders

(a) Create keyspace named Ecommerce

```
CREATE KEYSPACE Ecommerce WITH replication =  
[ 'class': 'SimpleStrategy', 'replication-factor': 1 ];
```

(b) Create table Customer-Orders with Order-Id (Primary Key),
Customer-Name, Product-Name, Quantity, Order-Date,

```
USE Ecommerce;
```

```
CREATE TABLE Customer-Orders (Order-Id int PRIMARY KEY,  
Customer-Id int text, Customer-Name text, Product-Name  
text, Quantity int, Order-Date  
date);
```

(c) Insert records

```
INSERT INTO Customer-Orders VALUES (101, 'Ravi', 'Laptop', 1,  
    (2026-02-20));
```

```
INSERT INTO Customer-Orders VALUES (102, 'Kiran', 'Mobile', 2,  
    (2026-02-21));
```

(d) Add new column Delivery-Status using ALTER

```
ALTER TABLE Customer-Orders ADD Delivery-Status text;
```

(e) Update delivery status for given order

```
UPDATE Customer-Orders SET Delivery-Status = 'Delivered'  
WHERE Order-Id = 101;
```

④ Employee Skills

(a) Create keyspace named Company

```
CREATE KEYSPACE Company WITH replication =  
    {'class': 'SimpleStrategy', 'replication_factor': 1};
```

(b) Create table Employee-Skills

```
CREATE Table TABLE Employee-Skills (Emp-Id int PRIMARY  
KEY, Emp-name text, skills set <text>);
```

(c) Insert records

```
INSERT INTO Table Employee-Skills (Emp-Id, Emp-Name, skills)  
VALUES (1, 'Anita', {'Java', 'Python'});
```

(d) Add a new skill to an existing employee

```
UPDATE Employee-Skills SET skills = skills + {'AWS'}  
WHERE Emp-Id = 1;
```

(e) Display

```
Table SELECT Emp-Name, skills FROM Employee-Skills;
```


⑤ Product Expiry using TTL

(a) ~~Exat~~ Create keyspace named store

```
CREATE KEYSPACE store WITH replication = {'class': 'SimpleStrategy', 'replication-factor': 1}
```

(b) Create table product-Info

```
CREATE TABLE product-Info (product-Id int PRIMARY KEY,  
product-Name text, Price float, Category text);
```

(c) Insert product record

```
INSERT INTO product-Info (product-Id, product-Name, Price,  
Category) VALUES (1, 'milk', 50, 'Dairy') USING  
TTL 20;
```

(d) Display product immediately

```
SELECT * FROM product-Info;
```

(e) Display products after 20 seconds

```
SELECT * FROM product-Info;
```

✓ 23/2/26

LAB 4

Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)

1. mkdir
2. ls

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC]
Starting resourcemanager
Starting nodemanagers
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -mkdir /bda_hadoop
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /
Found 1 items
drwxr-xr-x  - hadoop supergroup          0 2024-05-13 14:37 /bda_hadoop
```

3. put

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -put /home/hadoop/Desktop/bda_local.txt /bda_hadoop/file.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /bda_hadoop
Found 1 items
-rw-r--r--  1 hadoop supergroup          9 2024-05-13 14:42 /bda_hadoop/file.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -cat /bda_hadoop/file.txt
Hello!!!
```

4. copyFromLocal

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -copyFromLocal /home/hadoop/Desktop/bda_local.txt /bda_hadoop/file_cp_local.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -cat /bda_hadoop/file_cp_local.txt
Hello!!!
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$
```

5. get

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -get /bda_hadoop/file.txt /home/hadoop/Desktop/downloaded_file.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -getmerge /bda_hadoop/file.txt /bda_hadoop/file_cp_local.txt /home/hadoop/Desktop/downloaded_file.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -getfacl /bda_hadoop/
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -getfacl /bda_hadoop/
# file: /bda_hadoop
# owner: hadoop
# group: supergroup
user::rwx
group::r-x
other::r-x
```

6. copyToLocal

```
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -copyToLocal /bda_hadoop/file.txt /home/hadoop/Desktop
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -mv /bda_hadoop /abc
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /abc
Found 2 items
-rw-r--r-- 1 hadoop supergroup          9 2024-05-13 14:42 /abc/file.txt
-rw-r--r-- 1 hadoop supergroup          9 2024-05-13 14:52 /abc/file_cp_local.txt
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -cp /hello/ /hadoop_lab
cp: '/hello/': No such file or directory
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$
```

7. cat

```
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -cat /bda_hadoop/file_cp_local.txt
Hello!!!
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$
```

8. mv

```
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -mv /bda_hadoop /abc
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /abc
Found 2 items
-rw-r--r-- 1 hadoop supergroup          9 2024-05-13 14:42 /abc/file.txt
-rw-r--r-- 1 hadoop supergroup          9 2024-05-13 14:52 /abc/file_cp_local.txt
```

9. cp

```
-rw-r--r-- 1 hadoop supergroup          9 2024-05-13 14:52 /abc/file_cp_local.txt
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -cp /hello/ /hadoop_lab
cp: '/hello/': No such file or directory
hadoop@bmscece-HP-Elite-Tower-800-G9-Desktop-PC:~$
```


LAB - 5

HDFS commands:

1. mkdir — create directory

```
hdfs dfs -mkdir /bda-hadoop
hadoop fs -ls /
```

2. ls — List Directory Contents

```
hadoop fs -ls /
hadoop fs -ls /bda-hadoop
```

3. put — upload file to HDFS

```
hdfs dfs -put
/home/hduser/Desktop/bda-local.txt
/bda-hadoop/file.txt
hdfs dfs -cat /bda-hadoop/file.txt
```

4. copyFromLocal — Copy from local

```
hdfs dfs -copyFromLocal
hdfs dfs -cat
```

5. get / getmerge — Download from HDFS

6. copyToLocal — Copy To Local

7. cat — Display File Contents

8. mv — move / Rename

9. cp — Copy within HDFS

① Word Count MapReduce Program

wcMapper.java

```
public class WcMapper extends MapReduceBase implements Mapper
    <LongWritable, Text, Text, IntWritable> {
    public void map(LongWritable key, Text value, OutputCollector
        <Text, IntWritable> output, Reporter rep)
        throws IOException {
```


LAB 5

Implement WordCount Program on Hadoop framework

Word Count Program:

Mapper code: You have to copy paste this program into the WCMapper Java Class file

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;
public class WCMapper extends MapReduceBase implements Mapper<LongWritable, Text,
Text, IntWritable> {
    public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable> output,
Reporter rep) throws IOException {
        String line = value.toString();
        for (String word : line.split(" ")) {
            if (word.length() > 0) {
                output.collect(new Text(word), new IntWritable(1));
            }
        }
    }
}
```

Reducer Code: You have to copy paste this program into the WCReducer Java Class file

```
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;
public class WCReducer extends MapReduceBase implements Reducer<Text, IntWritable, Text,
IntWritable> {
    public void reduce(Text key, Iterator<IntWritable> value, OutputCollector<Text, IntWritable>
output, Reporter rep) throws IOException {
        int count = 0;
        while (value.hasNext()) {
            IntWritable i = value.next();
            count += i.get();
        }
    }
}
```

```

    }
    output.collect(key, new IntWritable(count));
}
}

```

Driver Code: You have to copy paste this program into the WCDriver Java Class file.

```

import java.io.IOException;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;
public class WCDriver extends Configured implements Tool {
    public int run(String[] args) throws IOException {
        if (args.length < 2) {
            System.out.println("Please give valid inputs");
            return -1;
        }
        JobConf conf = new JobConf(WCDriver.class);
        conf.setJobName("Word Count");
        FileInputFormat.setInputPaths(conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));
        conf.setMapperClass(WCMapper.class);
        conf.setReducerClass(WCReducer.class);
        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);
        JobClient.runJob(conf);
        return 0;
    }
    public static void main(String[] args) throws Exception {
        int exitCode = ToolRunner.run(new WCDriver(), args);
        System.out.println(exitCode);
    }
}

```

OUTPUT

```
C:\hadoop-3.3.0\sbin>hdfs dfs -cat /input_dir/input_file.txt
Hello World
Hello Hadoop
This is Hadoop test file
C:\hadoop-3.3.0\sbin>hadoop jar C:\MapReduceClient.jar wordcount /input_dir /output_dir
2021-04-24 15:24:57,242 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2021-04-24 15:24:57,714 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/Anusree/.staging/job_1619256355508_0002
2021-04-24 15:24:58,387 INFO input.FileInputFormat: Total input files to process : 1
2021-04-24 15:24:58,809 INFO mapreduce.JobSubmitter: number of splits:1
2021-04-24 15:24:59,255 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1619256355508_0002
2021-04-24 15:24:59,255 INFO mapreduce.JobSubmitter: Executing with tokens: []
2021-04-24 15:24:59,450 INFO conf.Configuration: resource-types.xml not found
2021-04-24 15:24:59,451 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2021-04-24 15:24:59,533 INFO impl.YarnClientImpl: Submitted application application_1619256355508_0002
2021-04-24 15:24:59,581 INFO mapreduce.Job: The url to track the job: http://LAPTOP-JG329ESD:8088/proxy/application_1619256355508_0002/
2021-04-24 15:24:59,582 INFO mapreduce.Job: Running job: job_1619256355508_0002
2021-04-24 15:25:12,857 INFO mapreduce.Job: Job job_1619256355508_0002 running in uber mode : false
2021-04-24 15:25:12,861 INFO mapreduce.Job: map 0% reduce 0%
2021-04-24 15:25:19,985 INFO mapreduce.Job: map 100% reduce 0%
2021-04-24 15:25:26,077 INFO mapreduce.Job: map 100% reduce 100%
2021-04-24 15:25:32,181 INFO mapreduce.Job: Job job_1619256355508_0002 completed successfully
2021-04-24 15:25:32,284 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=85
    FILE: Number of bytes written=530945
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=162
    HDFS: Number of bytes written=51
```

```
C:\hadoop-3.3.0\sbin>hdfs dfs -cat /output_dir/*
Hadoop 2
Hello 2
This 1
World 1
file 1
is 1
test 1

C:\hadoop-3.3.0\sbin>
```

LAB - 5

HDFS commands:

1. mkdir — create directory

```
hdfs dfs -mkdir /bda-hadoop  
hadoop fs -ls /
```

2. ls — List Directory Contents

```
hadoop fs -ls /  
hadoop fs -ls /bda-hadoop
```

3. put — upload file to HDFS

```
hdfs dfs -put  
/home/hduser/Desktop/bda-local.txt  
/bda-hadoop/file.txt  
hdfs dfs -cat /bda-hadoop/file.txt
```

4. copyFromLocal — copy from local

```
hdfs dfs -copyFromLocal  
hdfs dfs -cat
```

5. get / getmerge — Download from HDFS

6. copyToLocal — copy to local

7. cat — Display file contents

8. mv — move / rename

9. cp — copy within HDFS

① Word Count MapReduce Program

WcMapper.java

```
public class WcMapper extends MapReduceBase implements Mapper  
    <LongWritable, Text, Text, IntWritable> {
```

```
    public void map(LongWritable key, Text value, OutputCollector  
        <Text, IntWritable> output, Reporter rep)
```

```
        throws IOException {
```



```

String line = value.toString();
for (String word: line.split(" ")) {
    if (word.length() > 0) {
        output.collect(new Text(word), new
            IntWritable(1));
    }
}
}
}
}

WCReducer.java
public class WCReducer <Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterator <IntWritable> value,
        OutputCollector<Text, IntWritable> output, Reporter rep) throws IOException {
        int count = 0;
        while (value.hasNext()) {
            IntWritable i = value.next();
            count += i.get();
        }
        output.collect(key, new IntWritable(count));
    }
}
}
}

```

JAR File Creation Steps

1. Create new java project in Eclipse
2. Add Hadoop libraries to build path
3. Set java version
4. Create 3 java classes
5. Export as JAR File

9/3/26

Run Word Count MapReduce Job

1. Start Hadoop
2. Create input file
3. Upload to HDFS
4. Run MapReduce Job
5. View output

LAB 6

From the following link extract the weather data

<https://github.com/tomwhite/hadoop-Book/tree/master/input/ncdc/all>

Create a Map Reduce program to

a) find average temperature for each year from the NCDC data set.

AverageDriver

```
package temp;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class AverageDriver {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Please Enter the input and output parameters");
            System.exit(-1);
        }
        Job job = Job.getInstance();
        job.setJarByClass(AverageDriver.class);
        job.setJobName("Max temperature");
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        job.setMapperClass(AverageMapper.class);
        job.setReducerClass(AverageReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

AverageMapper

```
package temp;
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
public class AverageMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
```

```

    public static final int MISSING = 9999;
    public void map(LongWritable key, Text value, Context context) throws IOException,
    InterruptedException {
        int temperature;
        String line = value.toString();
        if (line.length() < 93) {
            return;
        }
        String year = line.substring(15, 19);
        if (line.charAt(87) == '+') {
            temperature = Integer.parseInt(line.substring(88, 92));
        } else {
            temperature = Integer.parseInt(line.substring(87, 92));
        }
        String quality = line.substring(92, 93);
        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(year), new IntWritable(temperature));
        }
    }
}

```

AverageReducer

```

package temp;
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
public class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {
        int sum = 0;
        int count = 0;
        for (IntWritable value : values) {
            sum += value.get();
            count++;
        }
        if (count > 0) {
            int average = sum / count;
            context.write(key, new IntWritable(average));
        }
    }
}

```

OUTPUT

```
~\hadoop-3.3.0\sbin>hadoop jar C:\avgtemp.jar temp.AverageDriver /input_dir/temp.txt /avgtemp_outputdir
2021-05-15 14:52:50,635 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2021-05-15 14:52:51,065 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2021-05-15 14:52:51,111 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/Anusree/.staging/job_1621060230696_0005
2021-05-15 14:52:51,735 INFO input.FileInputFormat: Total input files to process : 1
2021-05-15 14:52:52,751 INFO mapreduce.JobSubmitter: number of splits:1
2021-05-15 14:52:53,073 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1621060230696_0005
2021-05-15 14:52:53,073 INFO mapreduce.JobSubmitter: Executing with tokens: []
2021-05-15 14:52:53,237 INFO conf.Configuration: resource-types.xml not found
2021-05-15 14:52:53,238 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2021-05-15 14:52:53,312 INFO impl.YarnClientImpl: Submitted application application_1621060230696_0005
2021-05-15 14:52:53,352 INFO mapreduce.Job: The url to track the job: http://LAPTOP-76329ESD:8088/proxy/application_1621060230696_0005/
2021-05-15 14:52:53,353 INFO mapreduce.Job: Running job: job_1621060230696_0005
2021-05-15 14:53:06,640 INFO mapreduce.Job: Job job_1621060230696_0005 running in uber mode : false
2021-05-15 14:53:06,643 INFO mapreduce.Job:  map 0% reduce 0%
2021-05-15 14:53:12,758 INFO mapreduce.Job:  map 100% reduce 0%
2021-05-15 14:53:19,060 INFO mapreduce.Job:  map 100% reduce 100%
2021-05-15 14:53:25,967 INFO mapreduce.Job: Job job_1621060230696_0005 completed successfully
2021-05-15 14:53:26,096 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=72210
    FILE: Number of bytes written=674341
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=894860
    HDFS: Number of bytes written=8
    HDFS: Number of read operations=8
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
    HDFS: Number of bytes read erasure-coded=0
  Job Counters
    Launched map tasks=1
    Launched reduce tasks=1
    Data-local map tasks=1
    Total time spent by all nodes in occupied slots (ms)=3782
```

```
C:\hadoop-3.3.0\sbin>hdfs dfs -ls /avgtemp_outputdir
Found 2 items
-rw-r--r--  1 Anusree supergroup          0 2021-05-15 14:53 /avgtemp_outputdir/_SUCCESS
-rw-r--r--  1 Anusree supergroup          8 2021-05-15 14:53 /avgtemp_outputdir/part-r-000000

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /avgtemp_outputdir/part-r-000000
1901    46

C:\hadoop-3.3.0\sbin>
```

b) find the mean max temperature for every month

MeanMaxDriver.class

```
package meanmax;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MeanMaxDriver {

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
```



```

        System.err.println("Please enter the input and output parameters");
        System.exit(-1);
    }
    Job job = Job.getInstance();
    job.setJarByClass(MeanMaxDriver.class);
    job.setJobName("Mean and Max Temperature");
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    job.setMapperClass(MeanMaxMapper.class);
    job.setReducerClass(MeanMaxReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

MeanMaxMapper.class

```

package meanmax;
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
public class MeanMaxMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    public static final int MISSING = 9999;
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        int temperature;
        String line = value.toString();
        if (line.length() < 93) return;
        String month = line.substring(19, 21);    if (line.charAt(87) == '+') {
            temperature = Integer.parseInt(line.substring(88, 92));
        } else {
            temperature = Integer.parseInt(line.substring(87, 92));
        }
        String quality = line.substring(92, 93);
        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(month), new IntWritable(temperature));
        }
    }
}
}

```

MeanMaxReducer.class

```
package meanmax;
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
public class MeanMaxReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int maxTemp = 0;
        int totalTemp = 0;
        int count = 0;
        int days = 0;
        for (IntWritable value : values) {
            int temp = value.get();
            if (temp > maxTemp) {
                maxTemp = temp;
            }
            count++;
            if (count == 3) {
                totalTemp += maxTemp;
                maxTemp = 0;
                count = 0;
                days++;
            }
        }
        if (days > 0) {
            context.write(key, new IntWritable(totalTemp / days));
        } else {
            context.write(key, new IntWritable(0));
        }
    }
}
```

OUTPUT

```
C:\hadoop-3.3.0\sbin>hdfs dfs -cat /meanmax_output/*  
01      4  
02      0  
03      7  
04     44  
05    100  
06    168  
07    219  
08    198  
09    141  
10    100  
11     19  
12      3  
  
C:\hadoop-3.3.0\sbin>
```

LAB-6

PART-A - Average Temperature per year

Averagemapper.java

```
public class Averagemapper extends Mapper<LongWritable, Text, Text,
    IntWritable> {

    public static final int MISSING = 9999;

    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();
        String year = line.substring(15, 19);
        int temperature;
        if (line.charAt(87) == '+') {
            temperature = Integer.parseInt(line.substring(88, 92));
        } else {
            temperature = Integer.parseInt(line.substring(87, 92));
        }
        String quality = line.substring(92, 93);
        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(year), new IntWritable(temperature));
        }
    }
}
```

AverageReducer.java

```
public class AverageReducer extends Reducer<Text, IntWritable, Text,
    IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context
        context) throws IOException,
        InterruptedException {

        int sum = 0;
        int count = 0;
        for (IntWritable value: values) {
            sum += value.get();
            count++;
        }
    }
}
```



```

    }
    int average = sum / count;
    context.write (key, new IntWritable (average));
}
}

```

Output:

Q1	120	1901	46
Q2	113	1949	94
Q3	125	1950	3
Q4	130		
Q5	135		
Q6			
Q7			
Q8			
Q9			
Q10			
Q11			
Q12			

PART B - mean max Temperature Per month

meanmaxmapper.java

```

public class meanmaxMapper extends Mapper<LongWritable, Text, Text,
    IntWritable> {

```

```

    public static final int MISSING = 9999;

```

```

    public void map (LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

```

```

        String line = value.toString();

```

```

        String month = line.substring (19, 21);

```

```

        int temperature;

```

```

        if (line.charAt (87) == '+') {

```

```

            temperature = Integer.parseInt (line.substring
                (88, 92));

```

```

        }

```

```

        else {

```

```

            temperature = Integer.parseInt (line.substring
                (89, 92));

```

```

        }

```

```

        String quality = line.substring(92, 93);
        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(month), new IntWritable(
                temperature));
        }
    }
}

```

meanMaxReducer.java

```

public class meanMaxReducer extends Reducer<Text, IntWritable,
    Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {

```

```

        int sum = 0;
        int count = 0;
        for (IntWritable value : values) {
            sum += value.get();
            count++;
        }
        int mean = sum / count;
        context.write(key, new IntWritable(mean));
    }
}

```

Output:

01	44	12	117
02	17		
03	111		
04	194		
05	256		
06	278		
07	319		
08	283		
09	211		
10	156		
11	89		

LAB 7

For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words

TopNDriver.class

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;
public class TopN {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        String[] otherArgs = (new GenericOptionsParser(conf, args)).getRemainingArgs();
        if (otherArgs.length != 2) {
            System.err.println("Usage: TopN <in> <out>");
            System.exit(2);
        }
        Job job = Job.getInstance(conf);
        job.setJobName("Top N");
        job.setJarByClass(TopN.class);
        job.setMapperClass(TopNMapper.class);
        job.setReducerClass(TopNReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(otherArgs[0]));
        FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

TopNCombiner.class

```
package samples.topn;
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
```

```

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
public class TopNCombiner extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values)
            sum += val.get();
        context.write(key, new IntWritable(sum));
    }
}

```

TopNReducer.class

```

package samples.topn;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;
import java.util.LinkedHashMap;
import java.util.stream.Collectors;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
public class TopNReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private Map<Text, Integer> countMap = new HashMap<>();
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        countMap.put(new Text(key), sum);
    }
    protected void cleanup(Context context) throws IOException, InterruptedException {
        Map<Text, Integer> sortedMap = countMap.entrySet().stream().sorted(Map.Entry.<Text,
Integer>comparingByValue().reversed()).limit(20).collect(Collectors.toMap(
            Map.Entry::getKey,
            Map.Entry::getValue,
            (e1, e2) -> e1,
            LinkedHashMap::new));
        for (Map.Entry<Text, Integer> entry : sortedMap.entrySet()) {
            context.write(entry.getKey(), new IntWritable(entry.getValue()));
        }
    }
}

```

```
}
```

TopNMapper.class

```
package samples.topn;
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
public class TopNMapper extends Mapper<Object, Text, Text, IntWritable> {
    private static final IntWritable one = new IntWritable(1);
    private Text word = new Text();
    private String tokens = "[_#$%<>\\^=\\[\\]\\|\\*\\/\\\\\\,;\\.\\|-:()?!\"'"]";
    public void map(Object key, Text value, Mapper<Object, Text, Text, IntWritable>.Context
context)
        throws IOException, InterruptedException {
        String cleanLine = value.toString().toLowerCase().replaceAll(this.tokens, " ");
        StringTokenizer itr = new StringTokenizer(cleanLine);
        while (itr.hasMoreTokens()) {
            this.word.set(itr.nextToken().trim());
            context.write(this.word, one);
        }
    }
}
```

OUTPUT

```
C:\hadoop-3.3.0\sbin>jps
11072 DataNode
20528 Jps
5620 ResourceManager
45532 NodeManager
5140 NameNode

C:\hadoop-3.3.0\sbin>hdfs dfs -mkdir /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /
Found 1 items
drwxr-xr-x  - Anusree supergroup          0 2021-05-08 19:46 /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -copyFromLocal C:\input.txt /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /input_dir
Found 1 items
-rw-r--r--  1 Anusree supergroup          36 2021-05-08 19:48 /input_dir/input.txt

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /input_dir/input.txt
hello
world
hello
hadoop
bye
```



```

C:\hadoop-3.3.0\sbin>hadoop jar C:\sort.jar samples.topn.TopN /input_dir/input.txt /output_dir
2021-05-08 19:54:54,582 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2021-05-08 19:54:55,291 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/Anusree/.staging/job_1620483374279_0001
2021-05-08 19:54:55,821 INFO input.FileInputFormat: Total input files to process : 1
2021-05-08 19:54:56,281 INFO mapreduce.JobSubmitter: number of splits:1
2021-05-08 19:54:56,552 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1620483374279_0001
2021-05-08 19:54:56,552 INFO mapreduce.JobSubmitter: Executing with tokens: []
2021-05-08 19:54:56,843 INFO conf.Configuration: resource-types.xml not found
2021-05-08 19:54:56,843 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2021-05-08 19:54:57,387 INFO impl.YarnClientImpl: Submitted application application_1620483374279_0001
2021-05-08 19:54:57,587 INFO mapreduce.Job: The url to track the job: http://LAPTOP-JG329ESD:8088/proxy/application_1620483374279_0001/
2021-05-08 19:54:57,588 INFO mapreduce.Job: Running job: job_1620483374279_0001
2021-05-08 19:55:13,792 INFO mapreduce.Job: Job job_1620483374279_0001 running in uber mode : false
2021-05-08 19:55:13,794 INFO mapreduce.Job:  map 0% reduce 0%
2021-05-08 19:55:20,020 INFO mapreduce.Job:  map 100% reduce 0%
2021-05-08 19:55:27,116 INFO mapreduce.Job:  map 100% reduce 100%
2021-05-08 19:55:33,199 INFO mapreduce.Job: Job job_1620483374279_0001 completed successfully
2021-05-08 19:55:33,334 INFO mapreduce.Job: Counters: 54
    File System Counters
      FILE: Number of bytes read=65
      FILE: Number of bytes written=530397
      FILE: Number of read operations=0
      FILE: Number of large read operations=0
      FILE: Number of write operations=0
      HDFS: Number of bytes read=142
      HDFS: Number of bytes written=31
      HDFS: Number of read operations=8
      HDFS: Number of large read operations=0
      HDFS: Number of write operations=2
      HDFS: Number of bytes read erasure-coded=0

```

```

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /output_dir/*
hello 2
hadoop 1
world 1
bye 1

C:\hadoop-3.3.0\sbin>

```

LAB 7

Create MapReduce program to sort the content in alphabetical order listing only top 10 max occurrences of words

Mapper

```
public static class TopNMapper extends Mapper<Object, Text, Text,
    IntWritable> {

    private static final IntWritable one = new IntWritable(1);
    private Text word = new Text();
    private String tokens = "[_!$%#<>||^=||*^|||,;,-||=:()?!\"'"]";

    public void map(Object key, Text value, Mapper<Object, Text,
        Text, IntWritable> context
        context)
        throws IOException, InterruptedException {

        String cleanline = value.toString().toLowerCase().
            replaceAll(this.tokens, " ");

        StringTokenizer itr = new StringTokenizer(cleanline);
        while (itr.hasMoreTokens()) {
            this.word.set(itr.nextToken().trim());
            context.write(this.word, one);
        }
    }
}
```

Reducer

```
public class TopNReducer extends Reducer<Text, IntWritable, Text,
    IntWritable> {

    private Map<Text, IntWritable> countmap = new HashMap<>();

    public void reduce(Text key, Iterable<IntWritable> values,
        Reducer<Text, IntWritable, Text, IntWritable> context
        context) throws IOException, InterruptedException {
```

```

int sum = 0;
for (IntWritable val: values) {
    sum += val.get();
    this.countmap.put(new Text(key), new IntWritable(sum));
}

protected void cleanup(Reducer<Text, IntWritable, Text, IntWritable>
                        context context) throws IOException,
                        InterruptedException {

    Map<Text, IntWritable> sortedmap = MiscUtils.sortByValues
        (this.countmap);

    int counter = 0;
    for (Text key: sortedmap.keySet()) {
        if (counter++ == 20)
            break;
        context.write(key, sortedmap.get(key));
    }
}
}
}

```

Output :

Found 2 items
bms 2
college 2
computer
law 1
science 1

23/3/26

LAB 8 & 9

1. Write a Scala program to print numbers from 1 to 100 using for loop.

```
object ExampleForLoop1 {  
  def main(args: Array[String]) {  
    for(counter <- 1 to 100) {  
      print(counter + " ")  
    }  
    println()  
  }  
}
```

OUTPUT



```
bmscscse@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: $ nano ExampleForLoop1.scala  
bmscscse@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: $ scala -version  
Scala code runner version 2.11.12 -- Copyright 2002-2017, LAMP/EPFL  
bmscscse@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: $ nano ExampleForLoop1.scala  
bmscscse@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: $ scalac ExampleForLoop1.scala  
bmscscse@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: $ scala ExampleForLoop1  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71  
72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100  
bmscscse@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: $
```

2. Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.

```
from pyspark import SparkContext  
sc = SparkContext("local", "SimpleWordCount")  
  
# Read file  
rdd = sc.textFile("file.txt") # Replace with your filename  
  
# Count words  
counts = (rdd.flatMap(lambda line: line.split())  
           .map(lambda word: (word, 1))  
           .reduceByKey(lambda a, b: a + b)  
           .filter(lambda x: x[1] > 4))  
  
# Show result  
for word, count in counts.collect():  
    print(word, count)  
  
sc.stop()
```


OUTPUT

```
scala> val data = sc.textFile("/home/bmscece/Desktop/sparkdata.txt")
data: org.apache.spark.rdd.RDD[String] = /home/bmscece/Desktop/sparkdata.txt MapPartitionsRDD[9] at textFile at <console>:24

scala> val data = sc.textFile("/home/bmscece/Desktop/sparkdata.txt")
data: org.apache.spark.rdd.RDD[String] = /home/bmscece/Desktop/sparkdata.txt MapPartitionsRDD[11] at textFile at <console>:24

scala> val mapdata = splitdata.map(word => (word, 1))
mapdata: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[12] at map at <console>:25

scala> val reducedata = mapdata.reduceByKey(_ + _)
reducedata: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[13] at reduceByKey at <console>:25

scala> val filteredWords = reducedata.filter{ case (word, count) => count > 4 }
filteredWords: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[14] at filter at <console>:25

scala> filteredWords.collect.foreach(println)

scala> val data = sc.textFile("/home/bmscece/Desktop/sparkdata.txt")
data: org.apache.spark.rdd.RDD[String] = /home/bmscece/Desktop/sparkdata.txt MapPartitionsRDD[16] at textFile at <console>:24

scala> val splitdata = data.flatMap(line => line.split(" "))
splitdata: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[17] at flatMap at <console>:25

scala> val mapdata = splitdata.map(word => (word, 1))
mapdata: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[18] at map at <console>:25

scala> val reducedata = mapdata.reduceByKey(_ + _)
reducedata: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[19] at reduceByKey at <console>:25

scala> val filteredWords = reducedata.filter{ case (word, count) => count >= 4 }
filteredWords: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[20] at filter at <console>:25

scala> filteredWords.collect.foreach(println)
(hello,4)

scala> filteredWords.saveAsTextFile("/home/bmscece/Desktop/filtered_words")

scala> val sortedWords = reducedata.sortBy(_._2, ascending = false)
sortedWords: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[26] at sortBy at <console>:25

scala> sortedWords.collect.foreach(println)
(hello,4)
(spark,3)
(world,2)
(big,1)
(data,1)
(,1)

scala> val countWords = filteredWords.count()
countWords: Long = 1

scala> println(s"Number of words with count >= 4: $countWords")
Number of words with count >= 4: 1

scala> □
```


Lab 8

Write scala program to print numbers from 1 to 100 using for loop

Program :

object PrintNumbers {

def main (args: Array[String]) : Unit = {

for (i ← 1 to 100) {

print (i)

}

}

}

Output :

1
2
3
4
5
⋮
96
97
98
99
100

Lab 9

Using RDD and Flatmap count how many times each word appears in a file and write a list of words whose count is strictly greater than 4 using spark

Program:

```
val textFile = sc.textFile("/home/Desktop/wc.txt")
val counts = textFile.flatMap(line => line.split(" "))
                        • map(word => (word, 1))
                        • reduceByKey(_+_)
```

```
import scala.collection.immutable.ListMap
val sorted = ListMap(counts.collect().sortBy((a,b) => a._2 > b._2):_*)

println(sorted)
for ((k,v) <- sorted) {
  if (v > 4) {
    println(k + " -> " + v)
  }
}
```

Output: Input File (wc.txt)

```
hello world hello hello test world hello data test
hello world hello
```

hello -> 6

~~hello world~~

~~hello world~~

~~hello world~~

LAB 10

Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}
import org.apache.spark.streaming.StreamingContext._

object SimpleStreamingCleaner {
  def main(args: Array[String]): Unit = {
    // Create Spark configuration and StreamingContext
    val conf = new SparkConf().setMaster("local[2]").setAppName("StreamingTextCleaner")
    val ssc = new StreamingContext(conf, Seconds(5))

    // Connect to socket stream (e.g., using: `nc -lk 9999`)
    val lines = ssc.socketTextStream("localhost", 9999)

    // Define stop words
    val stopWords = Set("a", "an", "the", "is", "in", "on", "and", "of", "to", "this", "that")

    // Define basic lemmatization map (for demo only)
    val lemmaMap = Map("running" -> "run", "tests" -> "test", "files" -> "file", "words" ->
"word")

    // Clean the text stream
    val cleaned = lines.map(_._2.toLowerCase.trim) // lower case and trim
    .flatMap(_._2.split("\\s+")) // split into words
    .filter(word => !stopWords.contains(word)) // remove stop words
    .map(word => lemmaMap.getOrElse(word, word)) // basic lemmatization

    // Print the cleaned text
    cleaned.print()

    // Start the stream
    ssc.start()
    ssc.awaitTermination()
  }
}
```

OUTPUT

```
scala> import java.nio.file.{Files, Paths}
import java.nio.file.{Files, Paths}

scala>

scala> val path = "/home/bmscecse/Desktop/WC.txt"
path: String = /home/bmscecse/Desktop/WC.txt

scala> val exists = Files.exists(Paths.get(path))
exists: Boolean = true

scala> println(s"File exists? $exists")
File exists? true

scala> val textFile = sc.textFile("file:///home/bmscecse/Desktop/WC.txt")
textFile: org.apache.spark.rdd.RDD[String] = file:///home/bmscecse/Desktop/WC.txt MapPartitionsRDD[40] at textFile at <console>:26

scala>

scala> val words = textFile.flatMap(line => line.split("\\s+"))
words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[41] at flatMap at <console>:27

scala>

scala> val pairs = words.map(word => (word, 1))
pairs: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[42] at map at <console>:27

scala>

scala> val counts = pairs.reduceByKey(_ + _)
counts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[43] at reduceByKey at <console>:27

scala>

scala> counts.collect().foreach{ case (word, count) => println(s"$word: $count") }
this: 3
is: 2
some: 1
: 1
hello: 5
testing: 1
just: 1
file: 3
test: 2
world: 4
contains: 1
a: 1
for: 1
words: 1

scala> 
```

Lab 10

Write a simple program on streaming in spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc) and print the cleaned text on the screen

Program :

```
import org.apache.spark.sql.SparkSession
```

```
val spark = SparkSession.builder
```

```
  .appName("TextCleaningStream")
```

```
  .master("local[*]")
```

```
  .getOrCreate()
```

```
import spark.implicits._
```

```
val lines = spark.readStream
```

```
  .format("socket")
```

```
  .option("host", "localhost")
```

```
  .option("port", 9999)
```

```
  .load()
```

```
val words = lines.as[String]
```

```
val stopWords = Set("is", "the", "a", "an", "and", "to")
```

```
val cleaned = words.map { line =>
```

```
  val lower = line.toLowerCase
```

```
  val noSpace = lower.trim.replaceAll("||s+", " ")
```

```
  val noPunct = noSpace.replaceAll("[^a-zA-Z]", " ")
```

```
  val filtered = noPunct.split(" ")
```

```
    .filter(word => !stopWords.contains(word))
```

```
  val lemmatized = filtered.map {
```

```
    case w if w.endsWith("ing") => w.dropRight(3)
```

```
    case w if w.endsWith("ed") => w.dropRight(2)
```

```
    case w => w
```



```
}  
lemmatized.mkString(" ")  
}  
val query = cleaned.writeStream  
    .outputMode("append")  
    .format("console")  
    .start()  
query.awaitTermination()
```

Output :

terminal input : This is a sample TEXT !!
I am learning spark streaming and coding

output on console : this is sample text
i am learn spark stream cod

✓
22/4/16